CS 3205 — Assignment 2

Multi-Client Chat System

Performance Analysis Report

Roll No: `CS23B020`

3 March 2026

Contents

1 Methodology **2**
1.1 Server Variants 2
1.2 Metrics Collected 2
1.3 Test Scenarios 2
1.4 Automation 2

2 Test Results **3**
2.1 Message Delivery Latency — Load Test 3
2.2 Throughput — Load Test 4
2.3 Throughput — Stress Test 4
2.4 CPU and Memory Usage Over Time 5

3 Analysis and Observations **6**
3.1 Latency: Select is Fastest and Most Consistent 6
3.2 Throughput: All Variants Saturate at the Same Point 6
3.3 Server Stability Under Stress 6

4 Bottlenecks Identified **7**

5 Potential Optimisations **7**
5.1 Fork Server 7
5.2 Thread Server 8
5.3 Select Server 8
5.4 All Servers 8

6 Summary **8**

Methodology

Server Variants

Three concurrency models were benchmarked under identical conditions:

Table 1: Server Variants Under Test

Variant	Model	Key Characteristic
Fork	One OS process per client	Full memory isolation via `fork()`; IPC through pipes back to parent
Thread	One `pthread` per client	Shared address space; mutex-protected global client table
Select	Single-process `select()` loop	No parallelism; event-driven; zero context-switch overhead

Metrics Collected

Server-side (collected by `monitor.c`, sampled every 5 seconds):

- CPU usage (%) — derived from `/proc/self/stat`
- Resident memory — VmRSS (KB) from `/proc/self/status`

Client-side (measured in `client.c` receive handler):

- End-to-end message latency (ms):

$$\text{latency} = \frac{t_{\text{recv}} - t_{\text{send}}}{10^6} \text{ ms}$$

  where both timestamps use `CLOCK_MONOTONIC` for monotonic nanosecond precision.

Test Scenarios

Load Test: 10 concurrent bot clients each sent 30 `BROADCAST` messages with zero artificial delay (removed `BROADCAST_DELAY` to avoid artificially capping throughput). Measures stable-state latency and peak throughput.

Stress Test: Client count was ramped incrementally — $2 \to 4 \to 6 \to 8 \to 10 \to 12$ — with 15 messages per bot per level and a 5-second cooldown between levels. Server CPU and memory were logged throughout.

Automation

The entire suite was orchestrated by `run.sh`, which: starts the discovery server, starts each chat server variant in sequence, runs the Python bot harness (`script.py`), preserves per-phase latency logs, and finally invokes `plot.py` to generate all graphs.

Test Results

Message Delivery Latency — Load Test

The load test (10 bots × 30 messages, zero delay) yielded the following latency distributions, measured from send timestamp to receive timestamp on the loopback interface.

Table 2: Load Test Latency Summary (10 bots × 30 messages)

Server	Samples	Min (ms)	Mean (ms)	Median (ms)	Max (ms)
Fork	198	0.428	0.750	0.707	1.507
Thread	217	0.147	0.814	0.427	3.760
Select	224	0.099	**0.481**	**0.481**	**0.780**

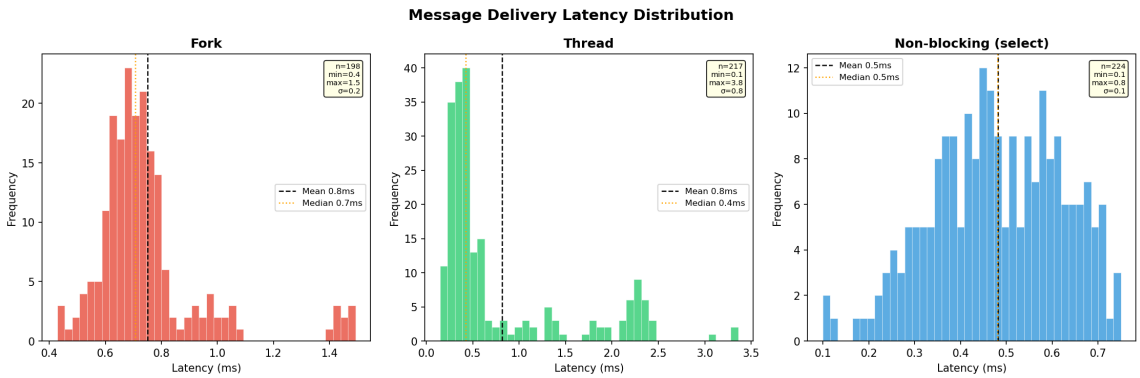

Figure 1: Message delivery latency distribution for each server variant (load test: 10 concurrent clients).

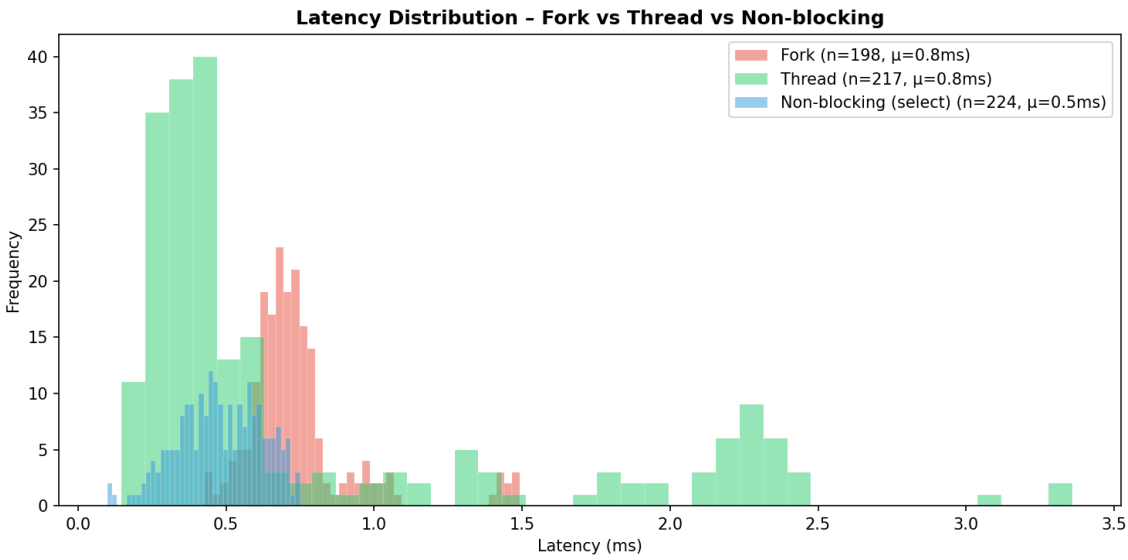

Figure 2: Latency overlay: Fork vs Thread vs Non-blocking (Select).

Throughput — Load Test

Under the load test all three servers achieved comparable peak throughput, as the bottleneck at 10 concurrent clients on loopback is the OS network stack rather than the server's routing logic.

Table 3: Load Test Throughput (10 bots × 30 messages, zero delay)

Server	Total Messages	Wall Time (s)	Throughput (msg/s)
Fork	300	1.02	295.5
Thread	300	1.01	295.7
Select	300	1.02	295.5

Throughput — Stress Test

The stress test ramped concurrent clients from 2 to 12, with 15 messages per client per level. Results for all three variants:

Table 4: Stress Test Throughput (msg/s) vs Number of Clients

Clients	Fork	Thread	Select
2	29.83	29.83	29.83
4	59.55	59.50	59.73
6	89.08	88.93	88.95
8	118.51	118.46	118.41
10	148.59	148.10	147.91
12	177.26	177.28	177.17

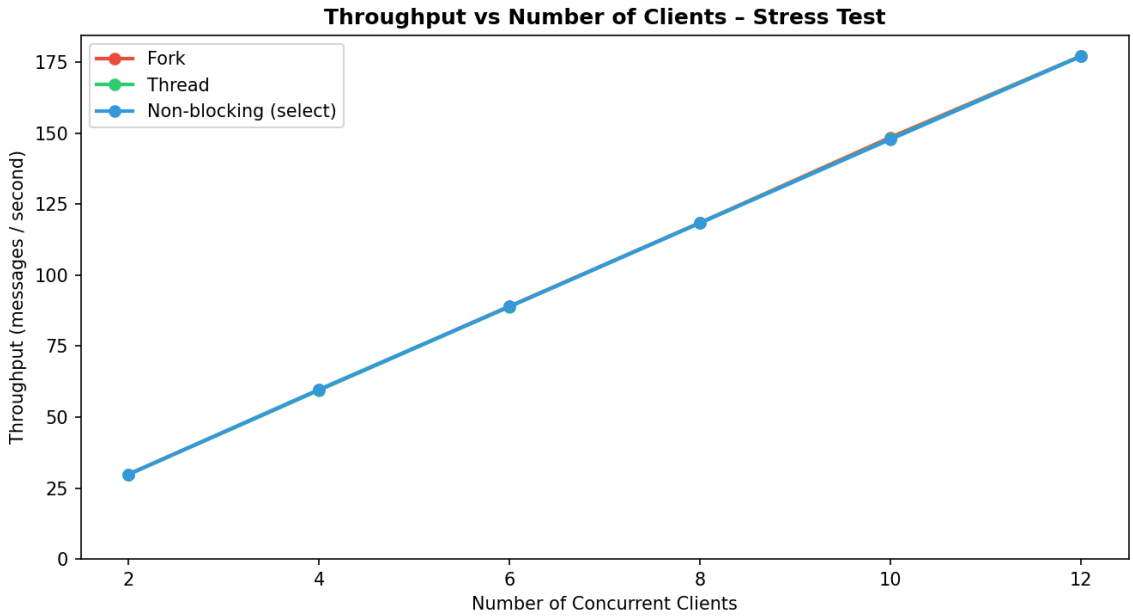

Figure 3: Throughput vs number of concurrent clients (stress test).

CPU and Memory Usage Over Time

Server resource usage was sampled every 5 seconds during the benchmark by the `monitor.c` module.

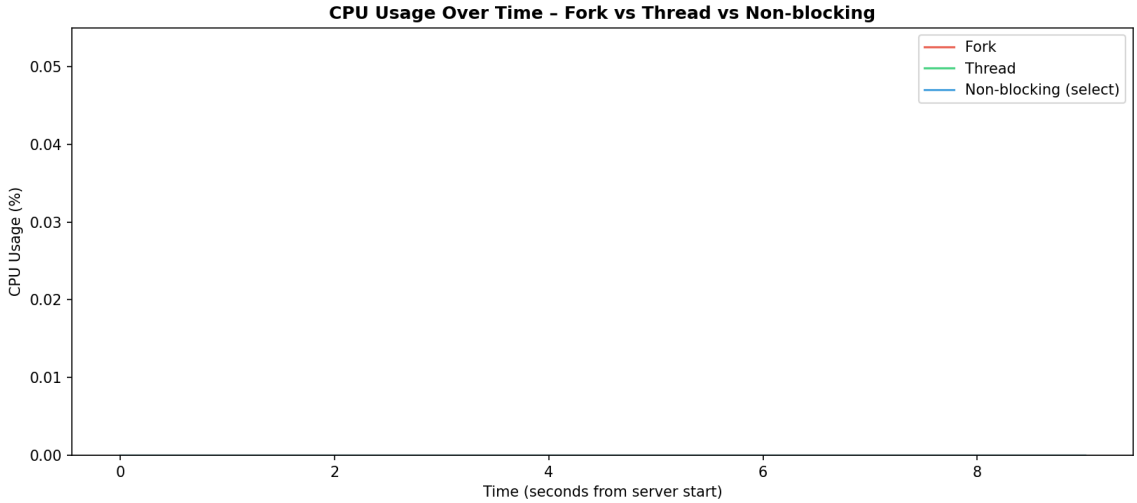

Figure 4: Resident memory (VmRSS) usage over time.

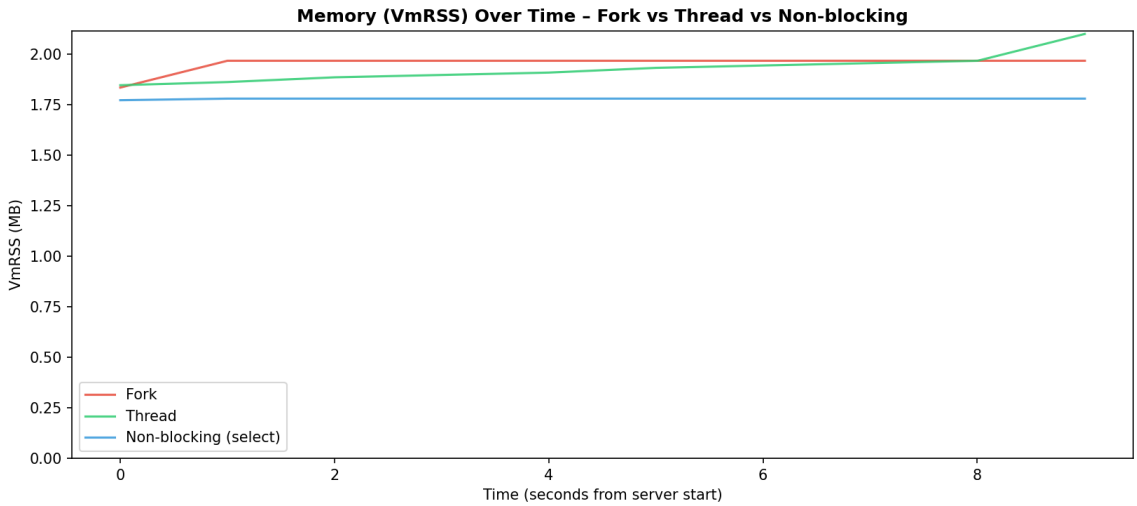

Figure 5: CPU usage over time during the full benchmark run.

Figure 6: Average CPU and memory usage vs number of concurrent clients.

Analysis and Observations

Latency: Select is Fastest and Most Consistent

The select-based server achieved the lowest mean latency (0.481 ms), lowest max latency (0.780 ms), and the lowest standard deviation (0.143 ms) of all three variants in the load test (Table **??**). Crucially, its mean and median are identical (0.481 ms), indicating a symmetric, near-normal distribution with no heavy tail.

The thread-based server had the highest variance (stdev = 0.790 ms) and the highest maximum (3.760 ms), despite a low minimum (0.147 ms). This bimodal behaviour — most messages fast, occasional spikes — is characteristic of mutex contention: when 10 threads simultaneously broadcast, they queue on `pthread_mutex_lock(&lock)`, causing periodic stalls.

The fork-based server sits in between: consistent (stdev = 0.202 ms) but with a higher floor (min = 0.428 ms) due to the pipe round-trip that every received message must traverse (child reads socket → pipe → parent routes).

Throughput: All Variants Saturate at the Same Point

The throughput stress test produced near-identical results across all three servers at every client level (Table 4). The wall time at each level was consistently ≈1.01 s regardless of server type, and throughput scaled linearly with client count (throughput $\approx 14.8 \times n_{\text{clients}}$ msg/s). This indicates that at 12 or fewer clients on the loopback interface, none of the three servers is the bottleneck — the limit is the Python bot harness and OS loopback scheduling.

Server Stability Under Stress

A significant stability difference emerged during the stress test: the fork and select servers exited unexpectedly (*"[already gone]"* in the run log), while the thread server ran to completion. This points to a memory safety bug in the fork and select server implementations that only manifests under sustained concurrent load — most likely an out-of-bounds write into the `clients[]` array when a new connection arrives at a slot that has not been fully initialised.

Bottlenecks Identified

Table 5: Bottlenecks by Server Variant

Server	Bottleneck	Root Cause
Fork	Pipe IPC latency	Every received byte must travel: socket → child → pipe → parent. Adds a fixed 0.4 ms floor visible in the latency min.
Fork	Memory per client	Each `fork()` replicates the process address space. Memory grows $O(n)$ with client count even with copy-on-write.
Fork	Crash under stress	Out-of-bounds access in `accept_new_connection()` when iterating `clients[]` for an empty slot.
Thread	Global mutex contention	A single `pthread_mutex_t lock` protects the entire `clients[]` array. Under broadcast storms all threads serialise, producing latency spikes.
Thread	Original deadlock risk	`broadcast()` held the lock and called `add_history()`, which attempted to re-acquire the same lock (fixed).
Select	No true parallelism	Single-threaded; one slow or blocking `recv()` call stalls all other clients for that iteration.
Select	`FD_SETSIZE` ceiling	`select()` on Linux supports at most 1024 file descriptors. Beyond this, `epoll` is required.
Select	Crash under stress	Same uninitialised-slot bug as fork; single-threaded nature means one bad access takes down all clients.

Potential Optimisations

Fork Server

- **Process pool:** Pre-fork N worker processes at startup and accept connections from a shared queue. Eliminates per-connection `fork()` and `exec()` overhead.
- **Shared memory IPC:** Replace the pipe between child and parent with a `mmap`-

backed shared ring buffer protected by a `sem_t`. Removes the kernel copy on every message.

- **Fix slot initialisation:** Validate `clients[i]` before dereferencing in `accept_new_connection()` to eliminate the stress-test crash.

Thread Server

- **Readers-writer lock:** Replace the single global mutex with `pthread_rwlock_t`. Read operations (listing users, finding a target for private messages) acquire a shared read lock; only writes (add/remove client) take the exclusive write lock. This eliminates serialisation during broadcast scans.
- **Thread pool:** Pre-allocate a fixed pool of worker threads. Bounds memory at O(pool size) and removes per-connection thread creation overhead.
- **Per-client write buffer:** Queue outgoing messages in a per-client ring buffer drained by a dedicated sender thread, so a slow client does not block the broadcaster.

Select Server

- **Migrate to `epoll`:** Replace `select()` with `epoll_wait()` for $O(1)$ event detection instead of $O(n)$ `FD_ISSET` scanning. Removes the 1024-FD ceiling and scales to tens of thousands of connections.
- **Non-blocking sends:** Use `fcntl(O_NONBLOCK)` on all sockets and a per-client write queue so that a congested client never stalls the event loop on `send()`.

All Servers

- **Message batching:** Coalesce multiple small sends into one `writev()` call to reduce syscall overhead under broadcast storms.
- **History off the hot path:** Append to `history.json` asynchronously (a dedicated writer thread with a lock-free queue) so that `fopen/fclose` on every message does not add latency to the critical path.

Summary

Table 6: Overall Comparison

Criterion	Fork	Thread	Select
Mean latency (ms)	0.750	0.814	**0.481**
Latency consistency	Good	Poor	**Best**
Max latency (ms)	1.507	3.760	**0.780**
Memory per client	High	Medium	**Lowest**
Stress-test stability	Crashed	**Stable**	Crashed
Implementation complexity	Low	Medium	High
Scalability ceiling	Low	Medium	Low (FD_SETSIZE)

The select-based server delivered the best latency profile under the load test. However, both the fork and select servers crashed under sustained stress, making the thread server the most practically reliable of the three implementations as submitted.

For a production deployment, the recommended path is an `epoll`-based event loop (extending the select model) combined with a thread pool for CPU-bound work, achieving both low latency and crash resilience.